Tarea 3 — Consultas y modelado de documentos en MongoDB

TI-4601 Bases de Datos Avanzadas · Fabricio Mena · Alisson Redondo · Christopher Rodriguez
Repositorio: https://github.com/fabrimena/TAREA-3-Bases-de-Datos-Avanzadas

1. Entorno y datos

MongoDB 8.0.34 de un solo nodo (`ti4601-tarea3`), sin conjunto de réplicas. La semilla del curso (`semilla: 2026`) generó 400 eventos con 1 189 zonas y `capacidad_total` 162 810 (`sha256 9fc6a2db...4774a`); `mongoimport` cargó los 400 documentos (`datos-manifest.txt`, `import.txt`). Todo se ejecuta con `consultas.js` como script y cada cifra sale de un archivo de `evidence/tarea3/`.

2. Parte A — Consultas

2.1. A1: conciertos de Heredia, diciembre a febrero

```
db.eventos.find({ categoria: "concierto", ciudad: "Heredia",
                  fecha: { $gte: "2026-12-01", $lte: "2027-02-28" } },
                { _id: 0, nombre: 1, recinto: 1, fecha: 1 }).sort({ fecha: 1 })
```

Devuelve **7 conciertos**, del *Concierto Urbano 331* (2026-12-02, Auditorio Norte) al *Concierto Solar 271* (2027-02-26, Parque de Eventos Sur) (`consultas.txt`). `fecha` es un string `AAAA-MM-DD`: su orden alfabético coincide con el cronológico, así que el rango y el `sort` sobre strings son correctos sin convertir a `Date`.

2.2. Índice compuesto y `explain` de A1

`db.eventos.createIndex({ categoria: 1, ciudad: 1, fecha: 1 })`. El script borra el índice antes de medir, así que el *antes* es siempre un recorrido completo.

	Etapas (raíz ← hoja)	totalKeysExamined	totalDocsExamined	nReturned
Antes	SORT ← PROJECTION_SIMPLE ← COLLSCAN	0	400	7
Después	PROJECTION_SIMPLE ← FETCH ← IXSCAN	7	7	7

Orden de los campos: igualdad → orden → rango. `categoria` y `ciudad` son igualdades: juntas reducen el índice a un solo tramo contiguo, (*concierto*, *Heredia*). Dentro de ese tramo las claves ya están ordenadas por `fecha`, así que el rango es un corte del tramo (`indexBounds ["2026-12-01", "2027-02-28"]`) y el `sort` sale gratis: desaparece la etapa `SORT`. Se examinan exactamente las 7 claves y los 7 documentos que se devuelven. Con la misma consulta y otros órdenes (`indice-orden.txt`):

Índice	Claves	Docs	Por qué
`{categoria, ciudad, fecha}` (propuesto)	7	7	las dos igualdades acotan; `fecha` corta y ordena
`{ciudad, categoria, fecha}`	7	7	mismas igualdades: el orden entre ellas no cambia el costo
`{categoria, fecha, ciudad}`	30	7	tras el rango, `ciudad` ya no acota: recorre los conciertos del rango
`{fecha, categoria, ciudad}`	90	7	recorre el rango de fechas (138 eventos), con saltos
`{categoria, fecha}` (Lab 3, paso 5)	31	31	no tiene `ciudad`: trae cada concierto del rango para filtrarlo

Entre las dos igualdades se puso `categoria` primero para que el prefijo `{categoria}` sirva a búsquedas solo por categoría. Con 400 documentos todo tarda 0–1 ms, por lo que la comparación se argumenta con claves y documentos examinados, no con tiempo.

2.3. A2: zona VIP con precio menor a 48 000 y comparación sin `$elemMatch`

```
db.eventos.find({zonas:{$elemMatch:{nombre:"VIP",precio:{$lt:48000}}}},
 { _id:0, nombre:1, zonas:{$elemMatch:{nombre:"VIP"}} })
```

La consulta devuelve **195 eventos** con zona VIP a menos de 48 000; la proyección muestra el nombre y únicamente el elemento VIP. Sin `$elemMatch`, al usar `zonas.nombre = "VIP"` y `zonas.precio < 48000` como condiciones separadas, aparecen **265 eventos**: **70 adicionales**. MongoDB puede satisfacer cada condición en zonas diferentes, por ejemplo VIP a 50 000 y General a 8 000. `$elemMatch` exige que ambas correspondan a una misma zona (resultados de `consultas.js`; evidencia final: `consultas.txt`).

2.4. A3 y A4

A3. Consulta de eventos con la etiqueta `accesible`, excluyendo las categorías teatro y conferencia:

```
db.eventos.find({etiquetas:"accesible",
 categoria:{$nin:["teatro","conferencia"]}}, {_id:0,nombre:1})
```

Resultado: **91 eventos**. La igualdad sobre `etiquetas` con el valor `accesible` comprueba si ese valor pertenece al arreglo; no necesita `$in` ni `$elemMatch`. `$nin` excluye las dos categorías.

A4. Eventos cuyo campo `patrocinador` no existe, agrupados por categoría:

```
db.eventos.aggregate([{$match:{patrocinador:{$exists:false}}},
 {$group:{_id:"$categoria",eventos:{$sum:1}}},{$sort:{_id:1}}])
```

Se obtienen **275 eventos**: concierto 57, conferencia 61, deporte 58, festival 49 y teatro 50. `$exists:false` selecciona documentos sin `patrocinador`, mientras que `patrocinador:null` incluye también los que tienen el campo con valor `null`. Como la semilla no genera ese valor, ambos filtros producen el mismo conteo.

2.5. A5: capacidad total por ciudad

```
db.eventos.aggregate([{$unwind:"$zonas"},
 {$group:{_id:"$ciudad",capacidad_total:{$sum:"$zonas.capacidad"}}},
 {$sort:{capacidad_total:-1}}])
```

`$unwind` produce un documento por cada elemento de `zonas`: un evento con 3 zonas pasa a ser 3 documentos que repiten `ciudad` y llevan una sola zona cada uno, así que los 400 eventos se convierten en 1 189 filas y `$sum` puede sumar `zonas.capacidad` por ciudad. Resultado (8 ciudades): **Heredia 30 380**, Liberia 24 160, San Carlos 21 810, Alajuela 21 020, Puntarenas 17 180, San José 17 150, Limón 15 560 y **Cartago 15 550**. Control: la suma de las 8 es **162 810**, igual a `capacidad_total` de `datos-manifest.txt`.

3. Parte B — Anidar o referenciar

B.1 Tamaño (`parte_b.js`, `parte-b.txt`). Una orden con `_id`, `evento_id`, `zona`, `cantidad`, `estado` y `creada_en` pesa **123 B**; el evento `EV-0001` sin órdenes, **508 B**. La estimación ingenua $(16\,777\,216 - 508)/123$ da 136 396 órdenes, pero dentro de un arreglo cada elemento paga además 1 byte de tipo y su índice como *string* con terminador (`"0"`…`"999"`): medido con 1 000 órdenes embebidas, cada una cuesta **127,89 B** (4,89 B extra), y el costo sube a 131 B cuando el índice llega a 6 dígitos. El máximo exacto es **128 914 órdenes**: con ellas el documento mide 16 777 146 B y con una más, 16 777 277 B (no cabe). Mucho antes del límite el problema es el peso: con 10 000 órdenes el evento pasa de 0,5 KB a 1,29 MB, y *cada* lectura del evento (listar, ver zonas) trae todas sus órdenes.

B.2 Concurrencia. MongoDB serializa las escrituras sobre un mismo documento: si cientos de compradores agregan su orden al mismo evento, WiredTiger detecta conflictos de escritura y las reintenta una tras otra, y el evento se vuelve un documento caliente. *Ventaja:* el descuento de stock y el registro de la orden ocurren en una sola actualización atómica, sin transacción multi-documento. *Desventaja:* contención en la venta de un evento popular, un documento que crece sin límite y cuesta más en cada lectura y en la replicación, y el tope de 16 MB. **Decisión:** referenciar (`ordenes` con `evento_id`, como en el Lab 3) y dejar en el evento solo el contador `zonas.disponibles`.

B.3 Regla. *Anidar lo que se lee siempre junto, pertenece a un solo padre y está acotado; referenciar lo que crece sin límite, se escribe en paralelo o se consulta por separado.* En un sistema de biblioteca, los autores y el ISBN de un libro se anidan en el libro (pocos, fijos y siempre se muestran con él); los préstamos se referencian en su propia colección con `libro_id`, porque se acumulan sin fin, los registran varios mostradores a la vez y se consultan por usuario o por fecha de vencimiento.